

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)
```

```
data = pd.read_csv('/content/drive/MyDrive/iot_ids_dataset.csv')
data.head()
```

	Flow_Duration	Tot_Fwd_Pkts	Tot_Bwd_Pkts	Fwd_Pkt_Len_Mean	Bwd_Pkt_Len_Mean	Protocol	SYN_Flag_Cnt	ACK_Flag_Cnt	Avg_Pkt_Size
0	0.139356	0.079720	0.172526	0.181311	0.049262	0.087239	0.135516	0.131887	0.139356
1	0.168857	0.159034	0.138915	0.190084	0.166819	0.109172	0.146863	0.099775	0.168857
2	0.114223	0.095386	0.195710	0.129842	0.123275	0.134288	0.154566	0.165951	0.114223
3	0.233655	0.107364	0.110517	0.098858	0.174035	0.207636	0.124128	0.105708	0.233655
4	0.161868	0.148796	0.102990	0.148016	0.149767	0.162556	0.170538	0.158863	0.161868

Étapes suivantes : [Générer du code avec data](#) [New interactive sheet](#)

```
print("Nombre de lignes et colonnes :")
print(data.shape)

print("\nInformations générales :")
print(data.info())

print("\nValeurs manquantes :")
print(data.isnull().sum())
```

Nombre de lignes et colonnes :
(1299, 11)

Informations générales :
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1299 entries, 0 to 1298
Data columns (total 11 columns):

#	Column	Non-Null Count	Dtype
0	Flow_Duration	1299 non-null	float64
1	Tot_Fwd_Pkts	1299 non-null	float64
2	Tot_Bwd_Pkts	1299 non-null	float64
3	Fwd_Pkt_Len_Mean	1299 non-null	float64
4	Bwd_Pkt_Len_Mean	1299 non-null	float64
5	Protocol	1299 non-null	float64
6	SYN_Flag_Cnt	1299 non-null	float64
7	ACK_Flag_Cnt	1299 non-null	float64
8	Avg_Pkt_Size	1299 non-null	float64
9	Pkt_Inter_Arrival_Mean	1299 non-null	float64
10	Label	1299 non-null	object

dtypes: float64(10), object(1)

memory usage: 111.8+ KB

None

Valeurs manquantes :

Flow_Duration	0
Tot_Fwd_Pkts	0
Tot_Bwd_Pkts	0
Fwd_Pkt_Len_Mean	0
Bwd_Pkt_Len_Mean	0
Protocol	0
SYN_Flag_Cnt	0
ACK_Flag_Cnt	0
Avg_Pkt_Size	0
Pkt_Inter_Arrival_Mean	0

```
Label
dtype: int64
```

```
data.describe()
```

	Flow_Duration	Tot_Fwd_Pkts	Tot_Bwd_Pkts	Fwd_Pkt_Len_Mean	Bwd_Pkt_Len_Mean	Protocol	SYN_Flag_Cnt	ACK_Flag_Cnt
count	1299.000000	1299.000000	1299.000000	1299.000000	1299.000000	1299.000000	1299.000000	1299.000000
mean	0.430391	0.387560	0.429501	0.418640	0.435437	0.444868	0.410278	0.443613
std	0.265033	0.224621	0.252148	0.241333	0.251884	0.254644	0.241555	0.250464
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	0.156756	0.155912	0.162895	0.167384	0.168047	0.178640	0.163017	0.180417
50%	0.428425	0.383071	0.425989	0.424165	0.427665	0.445185	0.402977	0.438890
75%	0.681887	0.589151	0.669445	0.632509	0.672664	0.674074	0.635609	0.670554
max	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000

```
print(data.dtypes)
```

```
Flow_Duration      float64
Tot_Fwd_Pkts       float64
Tot_Bwd_Pkts       float64
Fwd_Pkt_Len_Mean   float64
Bwd_Pkt_Len_Mean   float64
Protocol           float64
SYN_Flag_Cnt       float64
ACK_Flag_Cnt       float64
Avg_Pkt_Size       float64
Pkt_Inter_Arrival_Mean float64
Label              object
dtype: object
```

```
print(data['Label'].value_counts())
```

```
Label
Normal    433
DoS       433
Probe     433
Name: count, dtype: int64
```

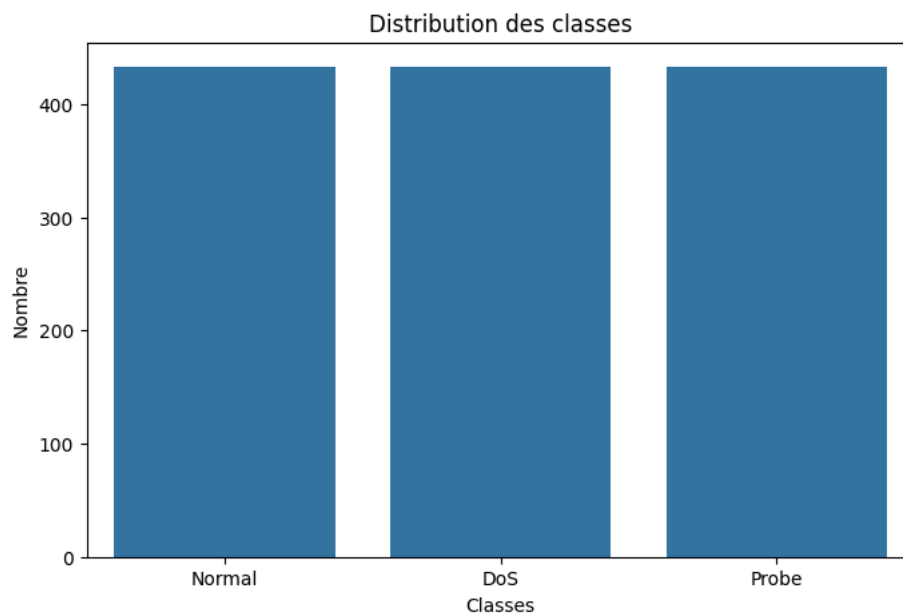
```
plt.figure(figsize=(8,5))

sns.countplot(x='Label', data=data)

plt.title("Distribution des classes")

plt.xlabel("Classes")
plt.ylabel("Nombre")

plt.show()
```

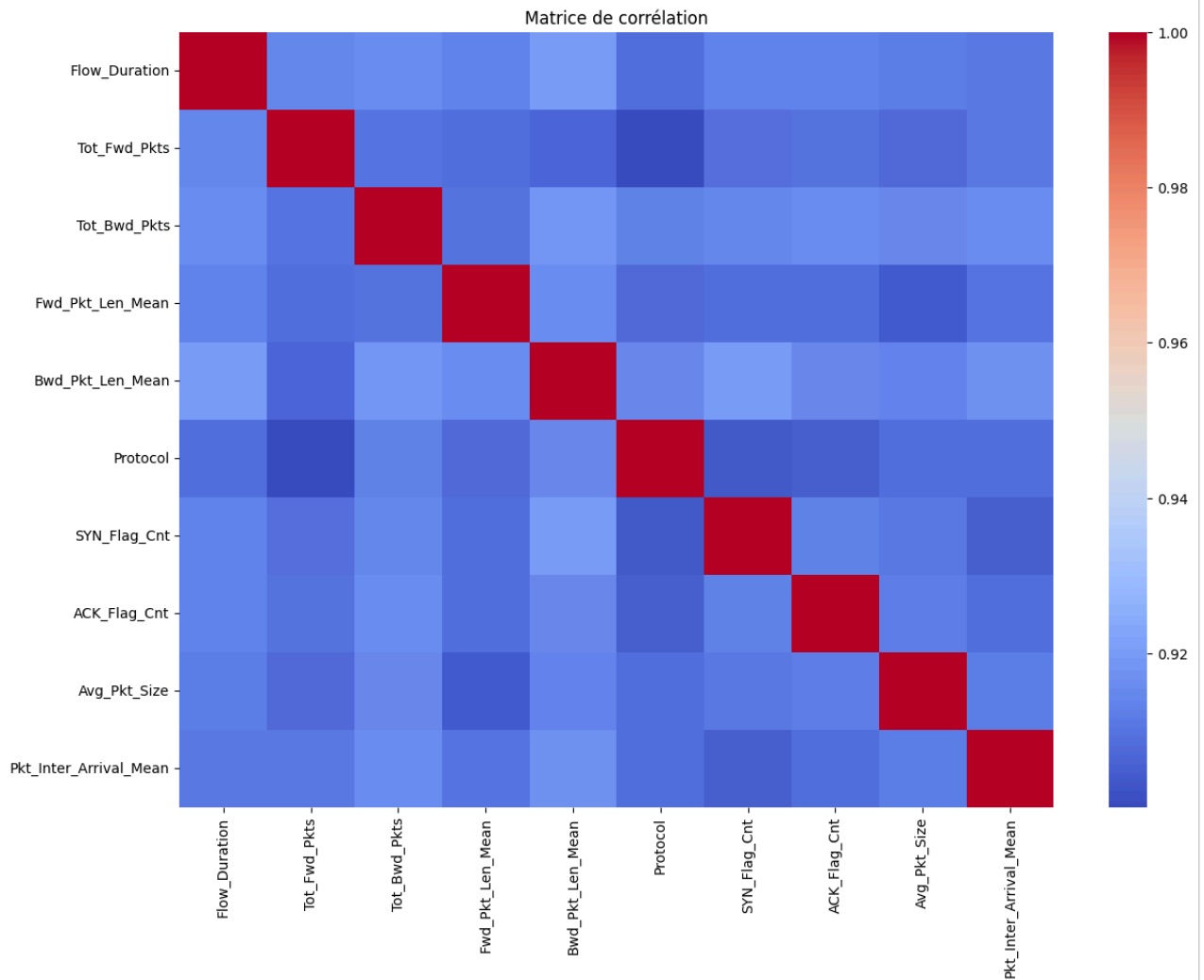


```
plt.figure(figsize=(14,10))

sns.heatmap(
    data corr(numeric_only=True),
    cmap='coolwarm',
    annot=False
)

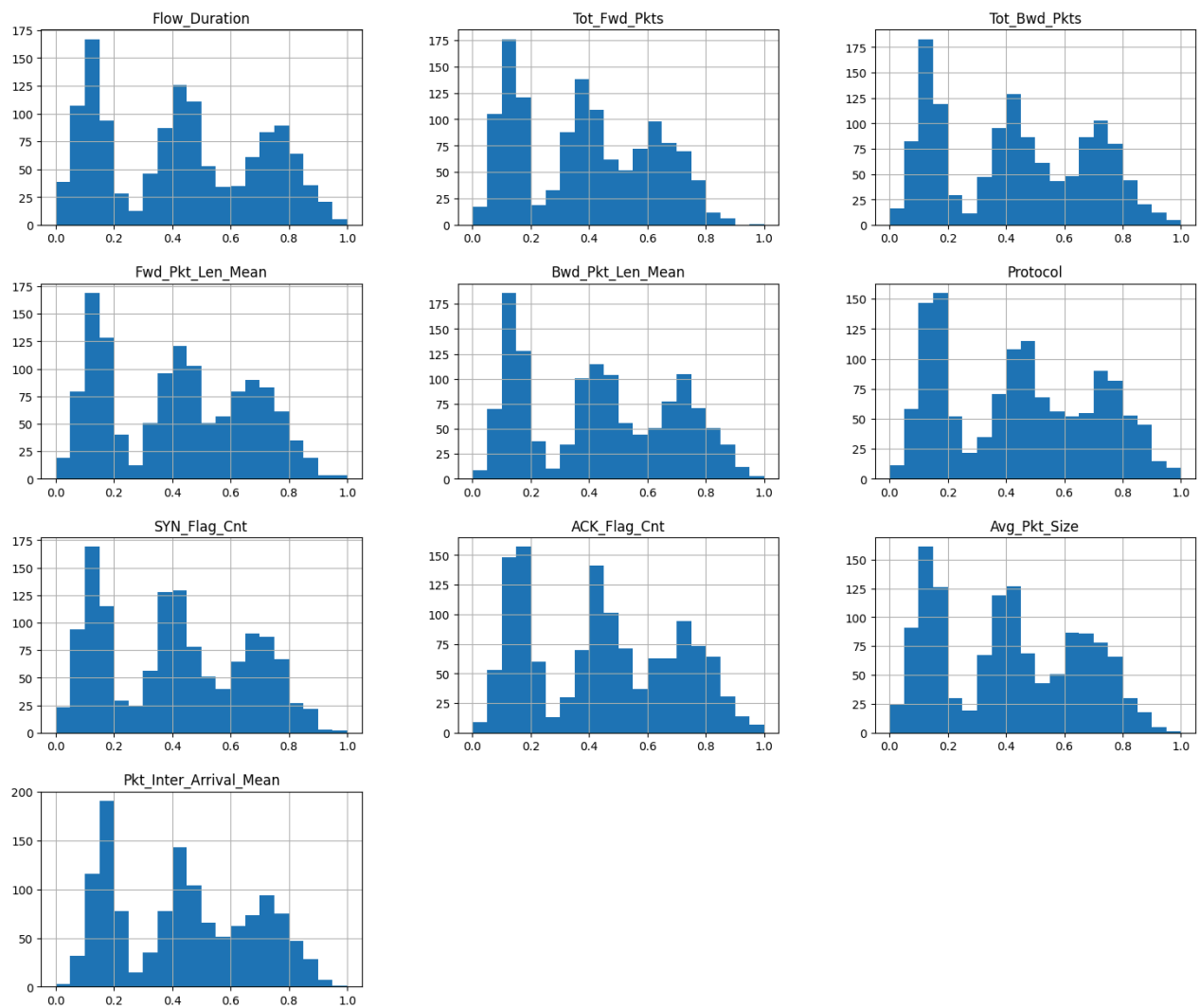
plt.title("Matrice de corrélation")

plt.show()
```



```
data.hist(
    figsize=(18,15),
    bins=20
)
plt.suptitle("Distribution des variables")
plt.show()
```

Distribution des variables



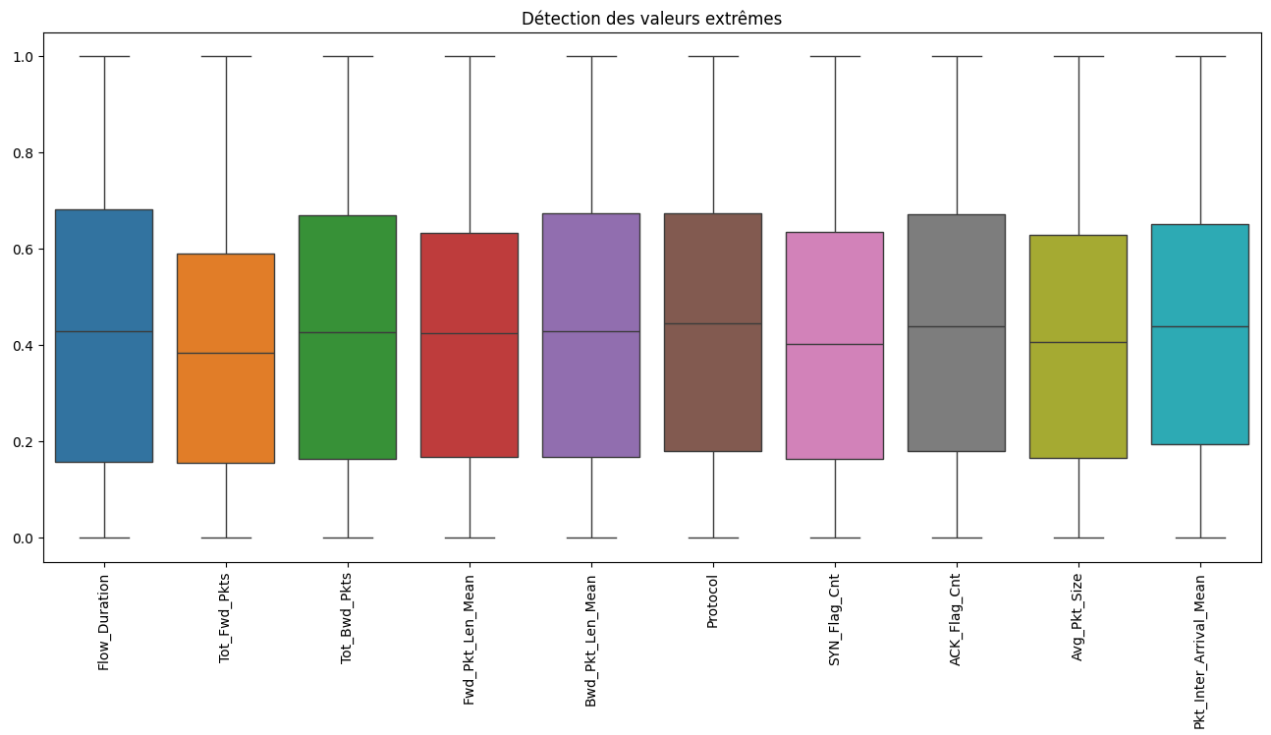
```
plt.figure(figsize=(16,7))

sns.boxplot(data=data.select_dtypes(include=np.number))

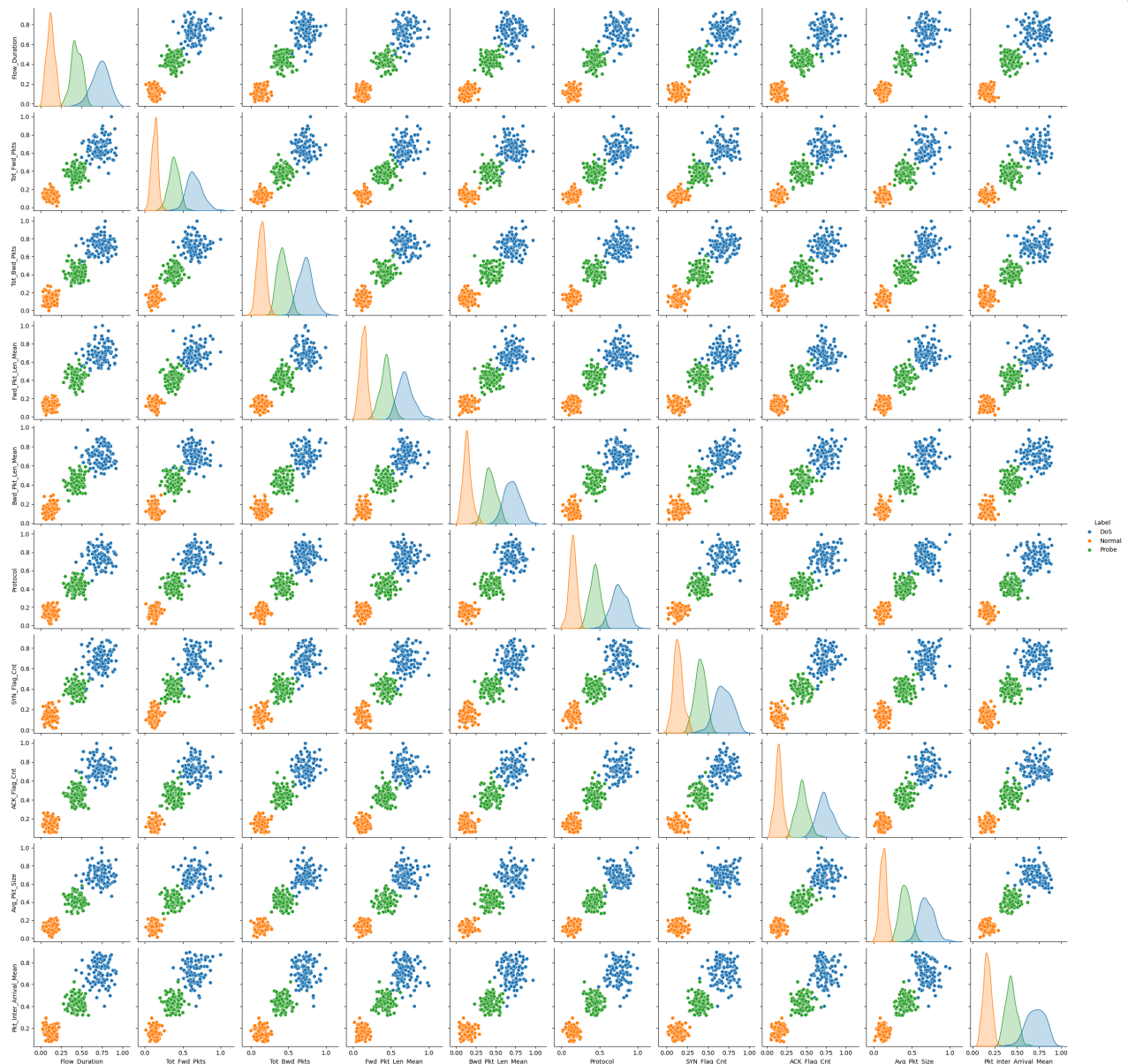
plt.xticks(rotation=90)

plt.title("Détection des valeurs extrêmes")

plt.show()
```



```
sample_data = data.sample(300)
sns.pairplot(
    sample_data,
    hue='Label'
)
plt.show()
```



```
encoder = LabelEncoder()
data['Label'] = encoder.fit_transform(data['Label'])
print(data['Label'].head())
```

```
0    1
1    1
2    1
3    1
4    1
Name: Label, dtype: int64
```

```
X = data.drop('Label', axis=1)
y = data['Label']
```

```
print("Variables d'entrée :")
print(X.head())
```

```
print("\nVariable cible :")
print(y.head())
```

```
Variables d'entrée :
Flow_Duration Tot_Fwd_Pkts Tot_Bwd_Pkts Fwd_Pkt_Len_Mean \
0    0.139356    0.079720    0.172526    0.181311
1    0.168857    0.159034    0.138915    0.190084
2    0.114223    0.095386    0.195710    0.129842
3    0.233655    0.107364    0.110517    0.098858
4    0.161868    0.148796    0.102990    0.148016
```

	Bwd_Pkt_Len_Mean	Protocol	SYN_Flag_Cnt	ACK_Flag_Cnt	Avg_Pkt_Size \
0	0.049262	0.087239	0.135516	0.131887	0.129235
1	0.166819	0.109172	0.146863	0.099775	0.171136
2	0.123275	0.134288	0.154566	0.165951	0.149339
3	0.174035	0.207636	0.124128	0.105708	0.091432
4	0.149767	0.162556	0.170538	0.158863	0.161797

	Pkt_Inter_Arrival_Mean
0	0.126689
1	0.163088
2	0.184876
3	0.194836
4	0.168413

Variable cible :

0	1
1	1
2	1
3	1
4	1

Name: Label, dtype: int64

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
print("Taille X_train :", X_train.shape)
print("Taille X_test :", X_test.shape)
```

Taille X_train : (1039, 10)
Taille X_test : (260, 10)

```
rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

rf_model.fit(X_train, y_train)
rf_pred = rf_model.predict(X_test)
rf_accuracy = accuracy_score(y_test, rf_pred)
print("Random Forest Accuracy :", rf_accuracy)
```

Random Forest Accuracy : 1.0

```
dt_model = DecisionTreeClassifier(
    random_state=42
)

dt_model.fit(X_train, y_train)
dt_pred = dt_model.predict(X_test)
dt_accuracy = accuracy_score(y_test, dt_pred)
print("Decision Tree Accuracy :", dt_accuracy)
```

Decision Tree Accuracy : 0.9923076923076923

```
lr_model = LogisticRegression(
    max_iter=1000
)

lr_model.fit(X_train, y_train)
lr_pred = lr_model.predict(X_test)
lr_accuracy = accuracy_score(y_test, lr_pred)
print("Logistic Regression Accuracy :", lr_accuracy)
```

Logistic Regression Accuracy : 1.0

```
knn_model = KNeighborsClassifier(
    n_neighbors=5
)

knn_model.fit(X_train, y_train)
knn_pred = knn_model.predict(X_test)
knn_accuracy = accuracy_score(y_test, knn_pred)
print("KNN Accuracy :", knn_accuracy)
```

KNN Accuracy : 1.0

```
results = pd.DataFrame({
    'Algorithme': [
```

```

        'Random Forest',
        'Decision Tree',
        'Logistic Regression',
        'KNN'
    ],

    'Accuracy': [
        rf_accuracy,
        dt_accuracy,
        lr_accuracy,
        knn_accuracy
    ]
})

print(results)

```

	Algorithme	Accuracy
0	Random Forest	1.000000
1	Decision Tree	0.992308
2	Logistic Regression	1.000000
3	KNN	1.000000

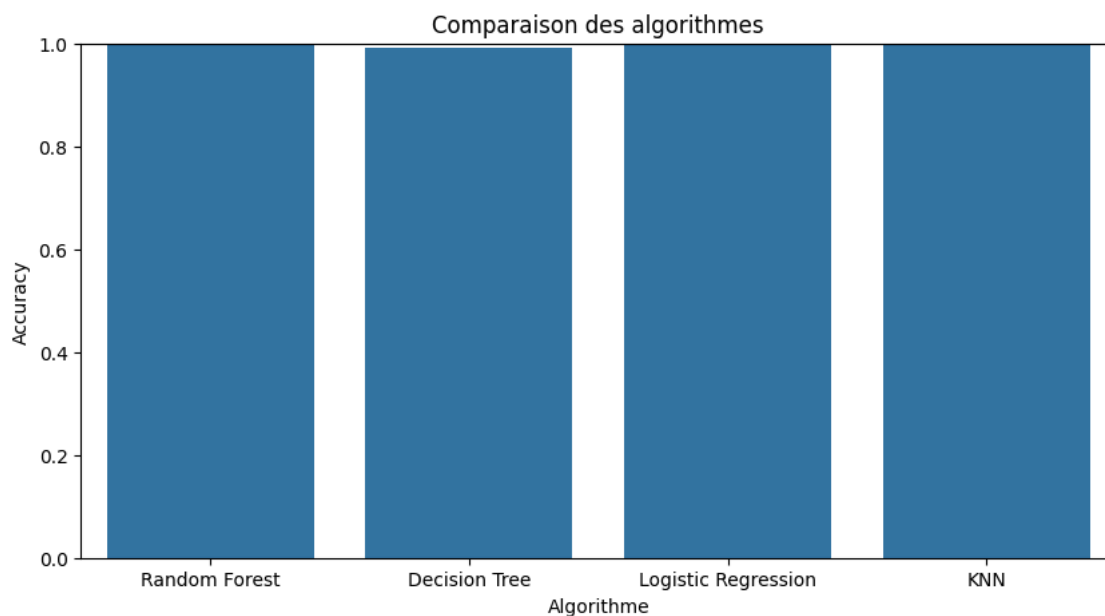
```

plt.figure(figsize=(10,5))

sns.barplot(
    x='Algorithme',
    y='Accuracy',
    data=results
)

plt.title("Comparaison des algorithmes")
plt.ylim(0,1)
plt.show()

```



```

best_model = results.loc[
    results['Accuracy'].idxmax()
]

print("Meilleur modèle :")
print(best_model)

```

```

Meilleur modèle :
Algorithme    Random Forest
Accuracy              1.0
Name: 0, dtype: object

```

```

cm = confusion_matrix(y_test, rf_pred)

print(cm)

```

```

[[90  0  0]
 [ 0 92  0]
 [ 0  0 78]]

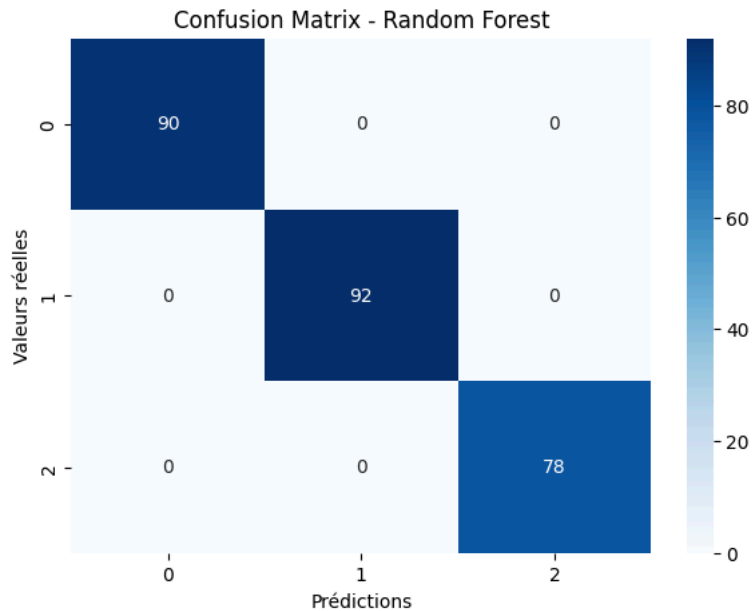
```



```
plt.figure(figsize=(7,5))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues'
)

plt.title("Confusion Matrix - Random Forest")
plt.xlabel("Prédictions")
plt.ylabel("Valeurs réelles")
plt.show()
```



```
report = classification_report(
    y_test,
    rf_pred
)
print(report)
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	90
1	1.00	1.00	1.00	92
2	1.00	1.00	1.00	78
accuracy			1.00	260
macro avg	1.00	1.00	1.00	260
weighted avg	1.00	1.00	1.00	260

```
importance = rf_model.feature_importances_
features = X.columns
importance_df = pd.DataFrame({
    'Feature': features,
    'Importance': importance
})
importance_df = importance_df.sort_values(
    by='Importance',
    ascending=False
)
print(importance_df)
```

	Feature	Importance
2	Tot_Bwd_Pkts	0.162243
0	Flow_Duration	0.147786
9	Pkt_Inter_Arrival_Mean	0.134623
4	Bwd_Pkt_Len_Mean	0.133172
7	ACK_Flag_Cnt	0.119394
8	Avg_Pkt_Size	0.089740
6	SYN_Flag_Cnt	0.069347
3	Fwd_Pkt_Len_Mean	0.061741
5	Protocol	0.043229
1	Tot_Fwd_Pkts	0.038724

```
plt.figure(figsize=(12,7))
```

```
plt.barh(
    importance_df['Feature'],
    importance_df['Importance']
)

plt.title("Importance des variables")
plt.xlabel("Importance")
plt.ylabel("Variables")
plt.show()
```

